

**Transmission Control Protocol
(TCP)
Error Control
Congestion Control**

TCP is a reliable transport layer protocol.

This means that an application program that delivers a stream of data to TCP relies on TCP to deliver the entire stream to the application program on the other end in order, without error, and without any part lost or duplicated.

Error control in TCP is achieved through the use of three tools:

checksum, acknowledgment, and time-out.

ReTransmission

The heart of error control mechanism is retransmission of segments

-when segment is sent, it is stored in a queue until it is acknowledged

Retransmission occurs

- When the retransmission timer expires
- When the sender receives three duplicates ACKs for the first segment in the queue

Acknowledgement Type

- ▶ Earlier, TCP used only one type of acknowledgement: More and more implementations are adding another type of acknowledgement
- ▶ Accumulative Acknowledgement (ACK), also namely accumulative positive acknowledgement
- ▶ Selective Acknowledgement (SACK), SACK is implemented as an option at the end of the TCP header.

- ❑ *Data may arrive out of order and be temporarily stored by the receiving TCP.*
- ❑ *TCP guarantees that no out-of-order data are delivered to the process.*

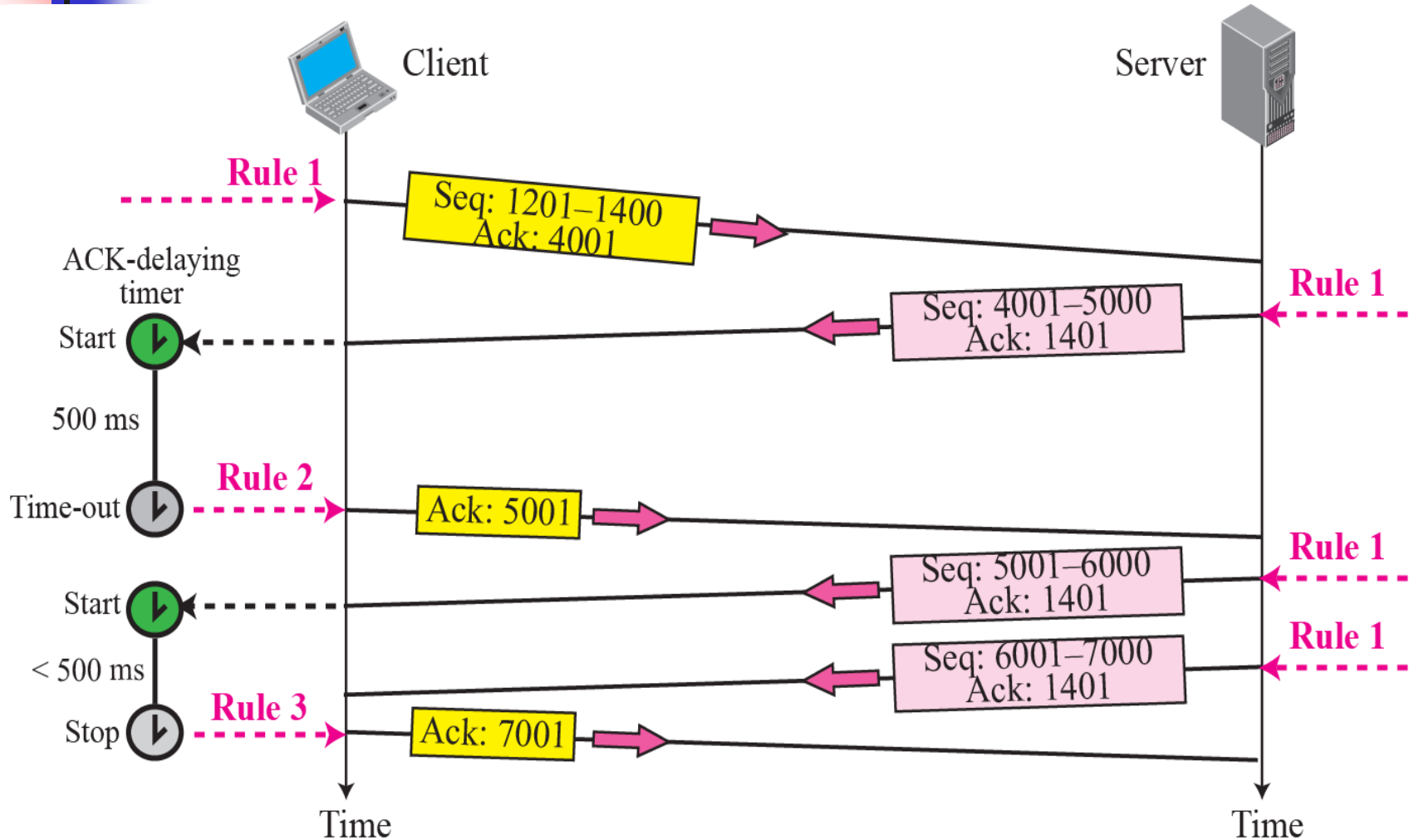
Rules for Generating ACK

1. When one end sends a data segment to the other end, it must include an ACK. That gives the next sequence number it expects to receive. (Piggyback)
2. The receiver needs to delay sending (until another segment arrives or 500ms) an ACK segment if there is only one outstanding in-order segment. It prevents ACK segments from creating extra traffic.
3. There should not be more than 2 in-order unacknowledged segments at any time. It prevent the unnecessary retransmission

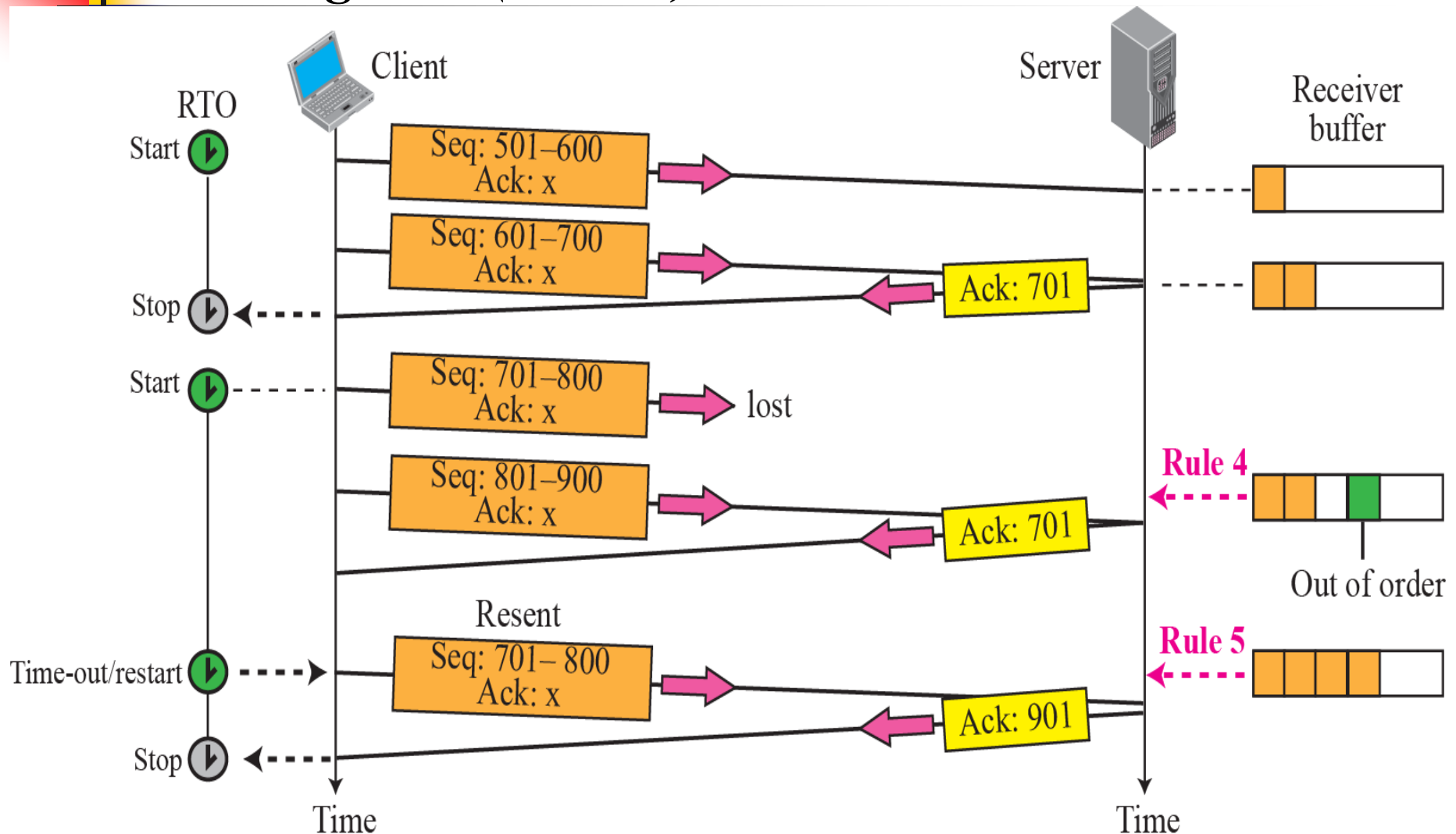
Rules for Generating ACK

4. When a segment arrives with an out-of-order sequence number that is higher than expected, the receiver immediately sends an ACK segment announcing the sequence number of the next expected segment. (for fast retransmission)
5. When a missing segment arrives, the receiver sends an ACK segment to announce the next sequence number expected.
6. If a duplicate segment arrives, the receiver immediately sends an ACK.

Normal operation

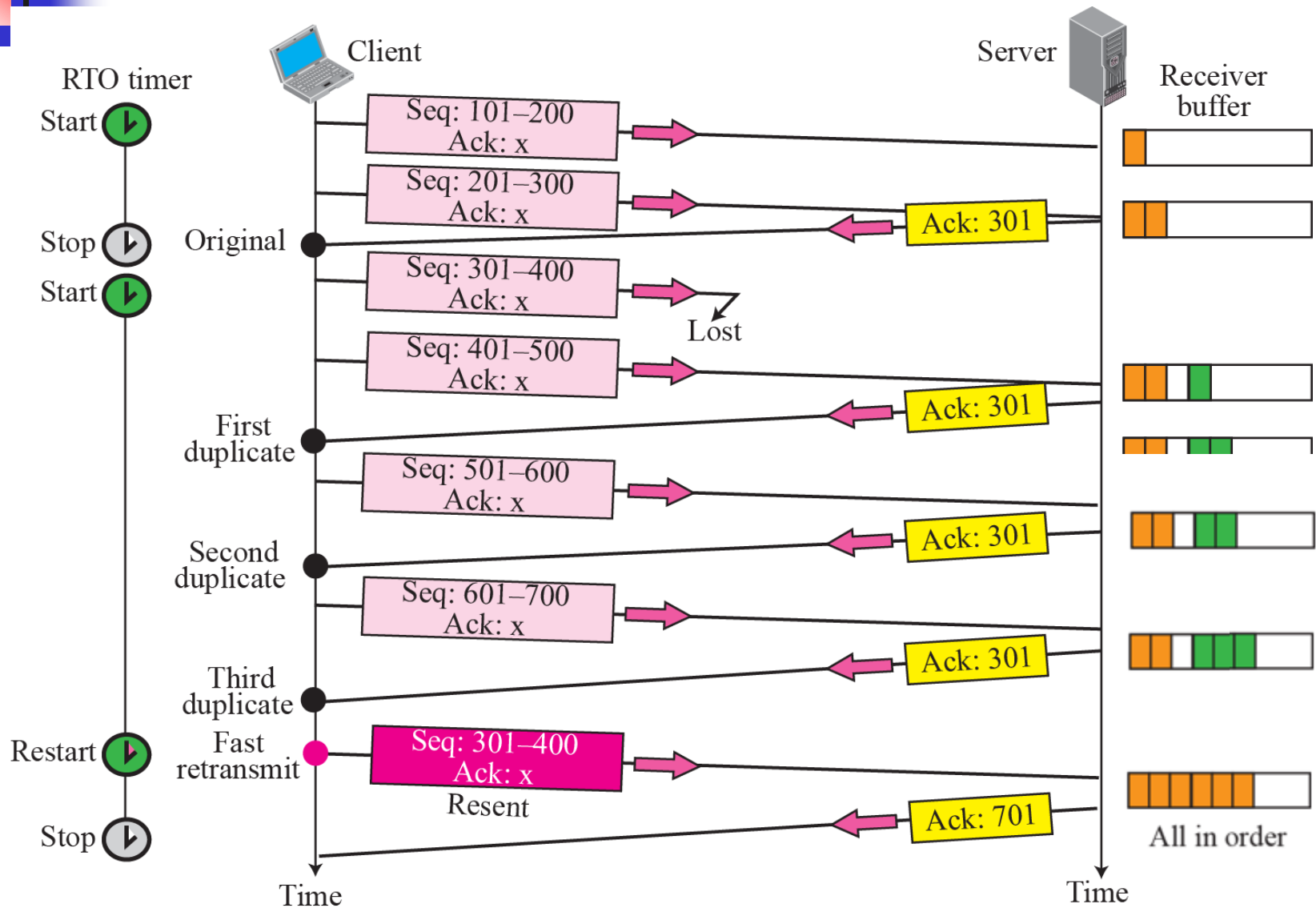


Lost segment (ACK-5)

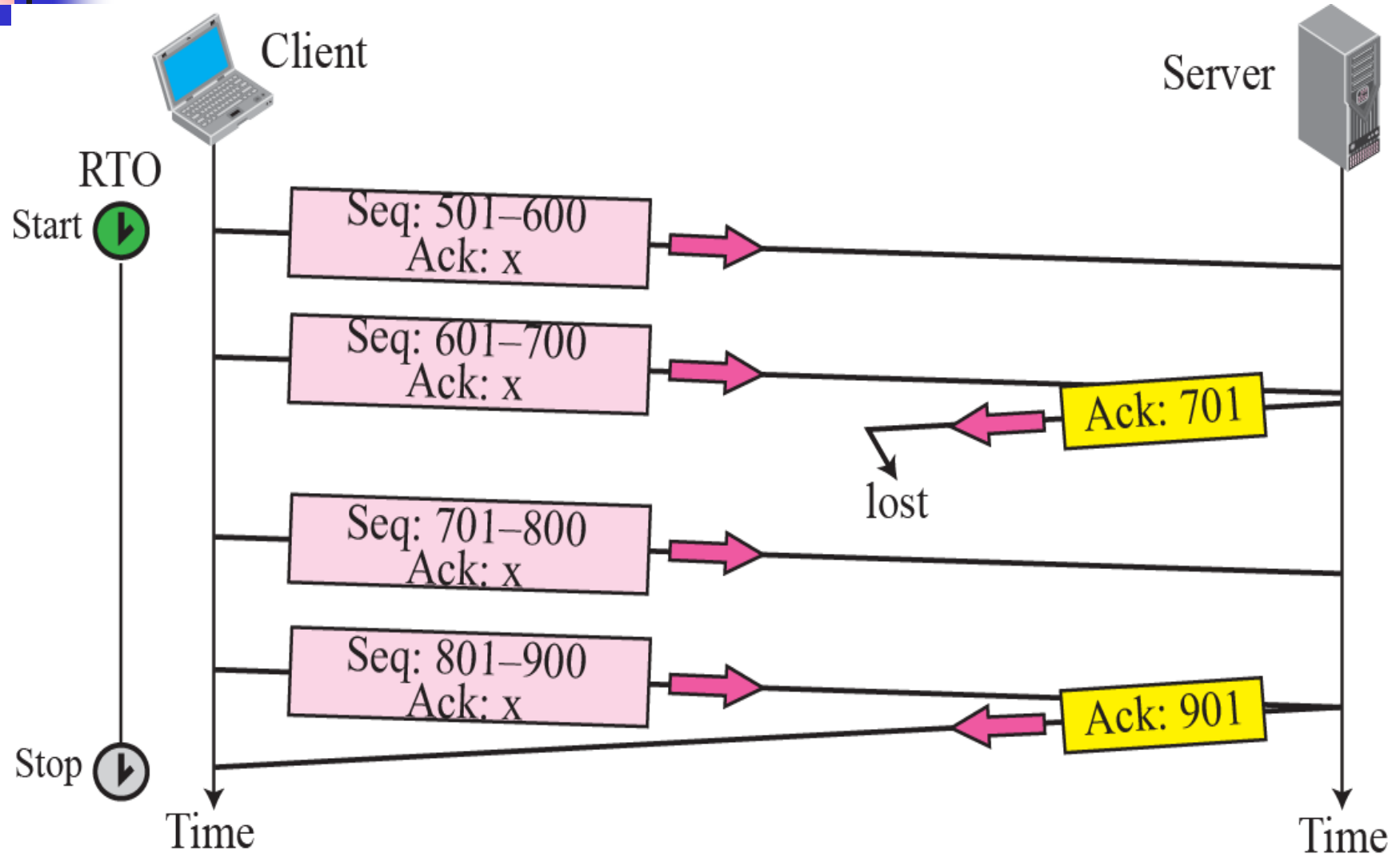


The receiver TCP delivers only ordered data to the process.

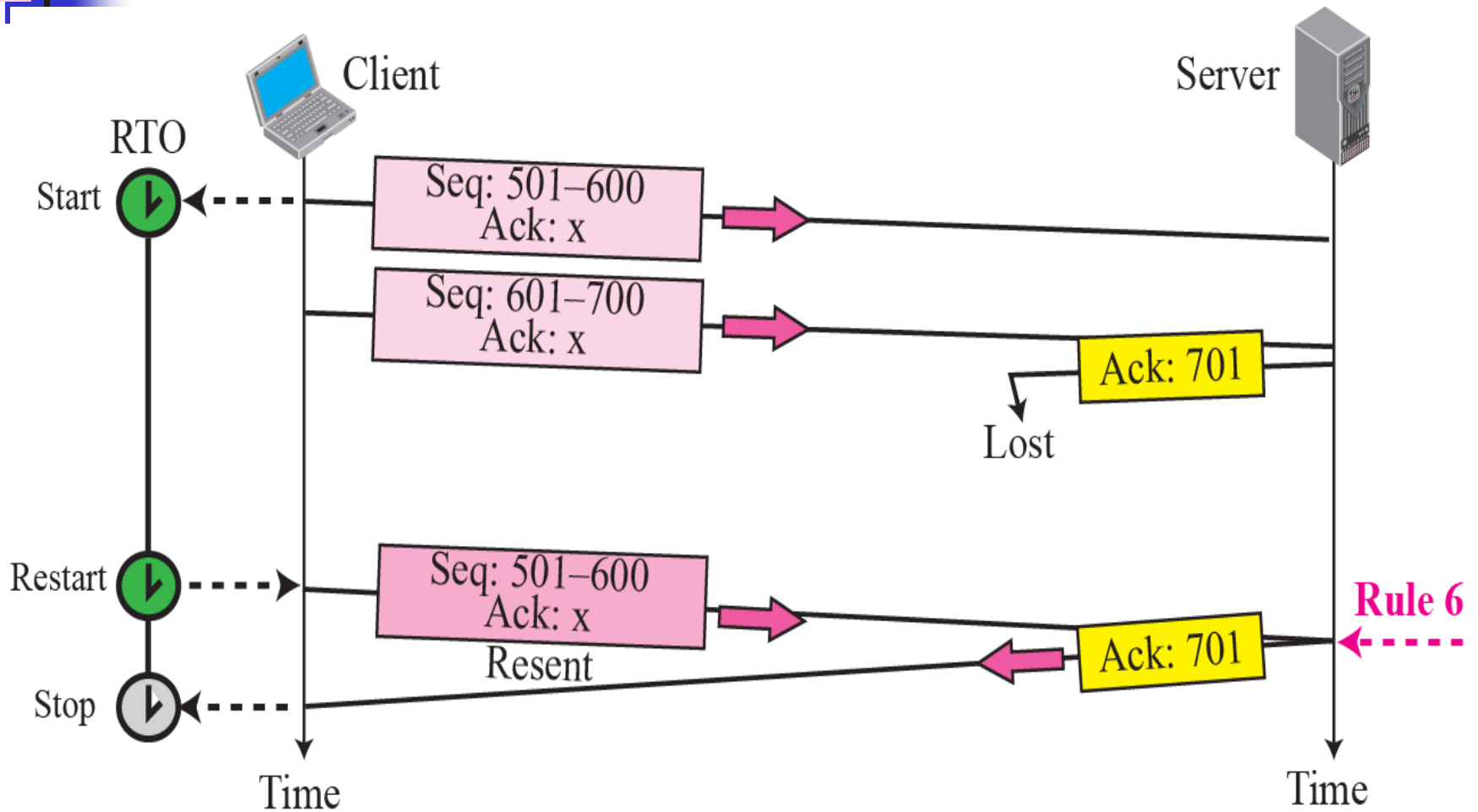
Fast retransmission (ACK-4)



Lost acknowledgment



(Duplicate Segment – ACK-6)



Lost acknowledgment corrected by resending a segment

Note

ACK segments do not consume sequence numbers and are not acknowledged.

Lost acknowledgments may create deadlock if they are not properly handled.

Congestion Control

Congestion in a network may occur if the **load** on the network is greater than the *capacity* of the network

i.e. the number of packets sent to the network—is greater than number of packets a network can handle.

Congestion happens in any system that involves waiting. It occurs because routers and switches have queues—buffers that hold the packets before and after processing.

Congestion control refers to techniques and mechanisms that can either

- prevent congestion, before it happens, or
- remove congestion, after it has happened.

Open-loop congestion control (prevent congestion)

policies are applied to prevent congestion before it happens. In these mechanisms, congestion control is handled by either the source or the destination.

Closed-Loop Congestion Control (remove congestion)

A mechanisms try to alleviate congestion after it happens. Several mechanisms have been used by different protocols.

Open-Loop Congestion Control

Retransmission Policy.

Retransmission in general may increase congestion in the network.

Good retransmission policy and the retransmission timers must be designed to optimize efficiency and at the same time prevent congestion.

Acknowledgment Policy

the acknowledgments are also part of the load in a network.

Sending fewer acknowledgments means imposing less load on the network.

Closed-Loop Congestion Control

Window Policy The type of window at the sender may also affect congestion.

One factor to determine the sender window size is the congestion in the Internet.

- ✓ sending transport layer monitor the congestion in Internet, by watching the lost packets, and
- ✓ use strategy to decrease the window size if congestion is increasing and vice versa.

CONGESTION CONTROL POLICY

TCP uses a congestion window and a congestion policy that avoid congestion and detect and alleviate congestion after it has occurred.

Congestion Window ($W_c - cwnd$)

We know sender window size is determined by the availability of buffer space at the receiver($rwnd$)

Other entity to determine sender window→ the network (also important)

If the network cannot deliver the data as fast as it is created by the sender, it must tell the sender to slow down.

Therefore, sender window size should be governed as follows :

Actual window size = minimum($rwnd$, $cwnd$)

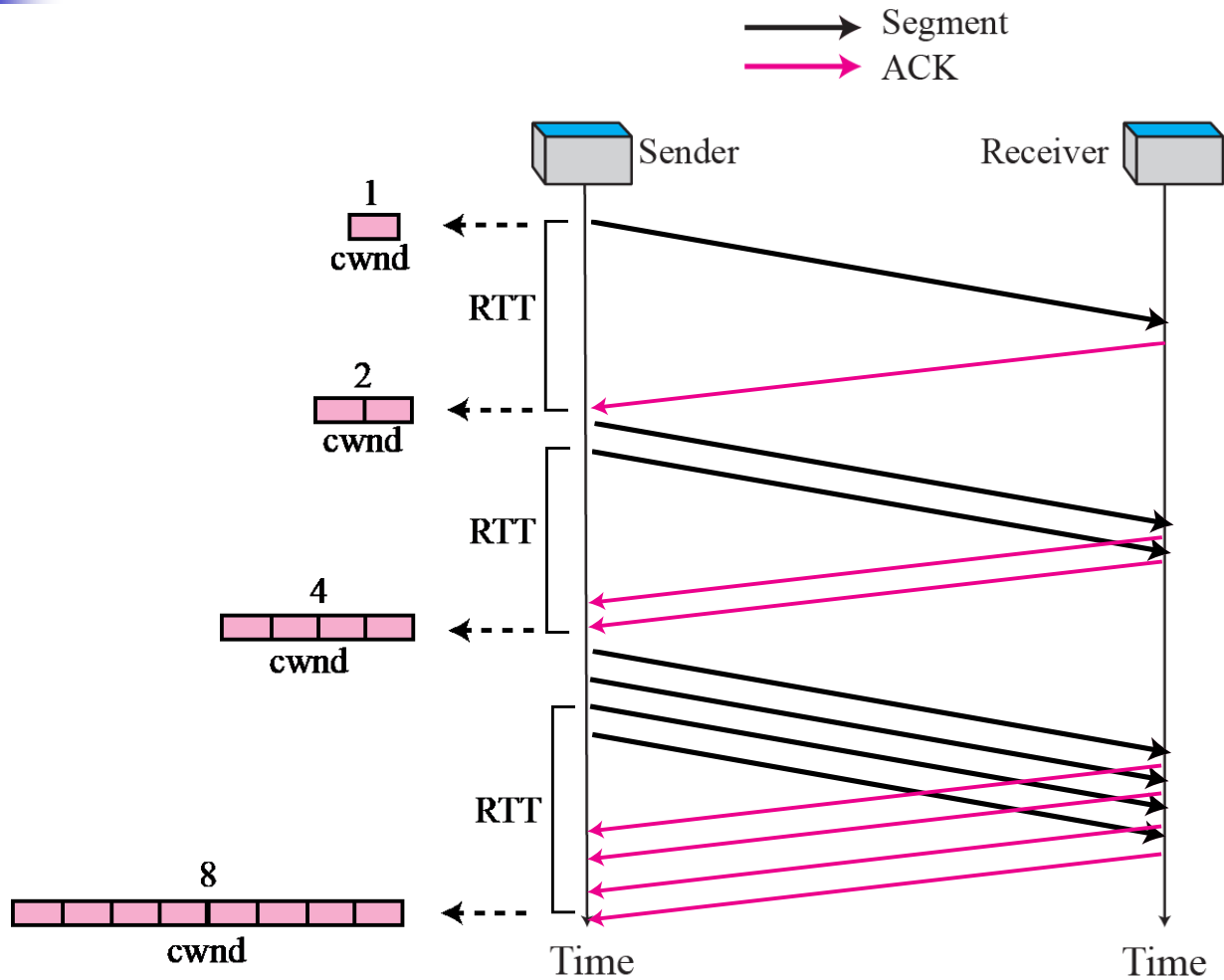
How the size of the congestion window W_c is determined ?

Congestion policy

TCP handles congestion based on three phases

1. Slow start – exponential increase
2. Congestion avoidance – Additive increase
3. Congestion detection – multiplicative decrease

Slow start, exponential increase



In the slow start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

Start

→ $\text{cwnd} = 1$

After 1 RTT

→ $\text{cwnd} = 1 \times 2 = 2 \rightarrow 2^1$

After 2 RTT

→ $\text{cwnd} = 2 \times 2 = 4 \rightarrow 2^2$

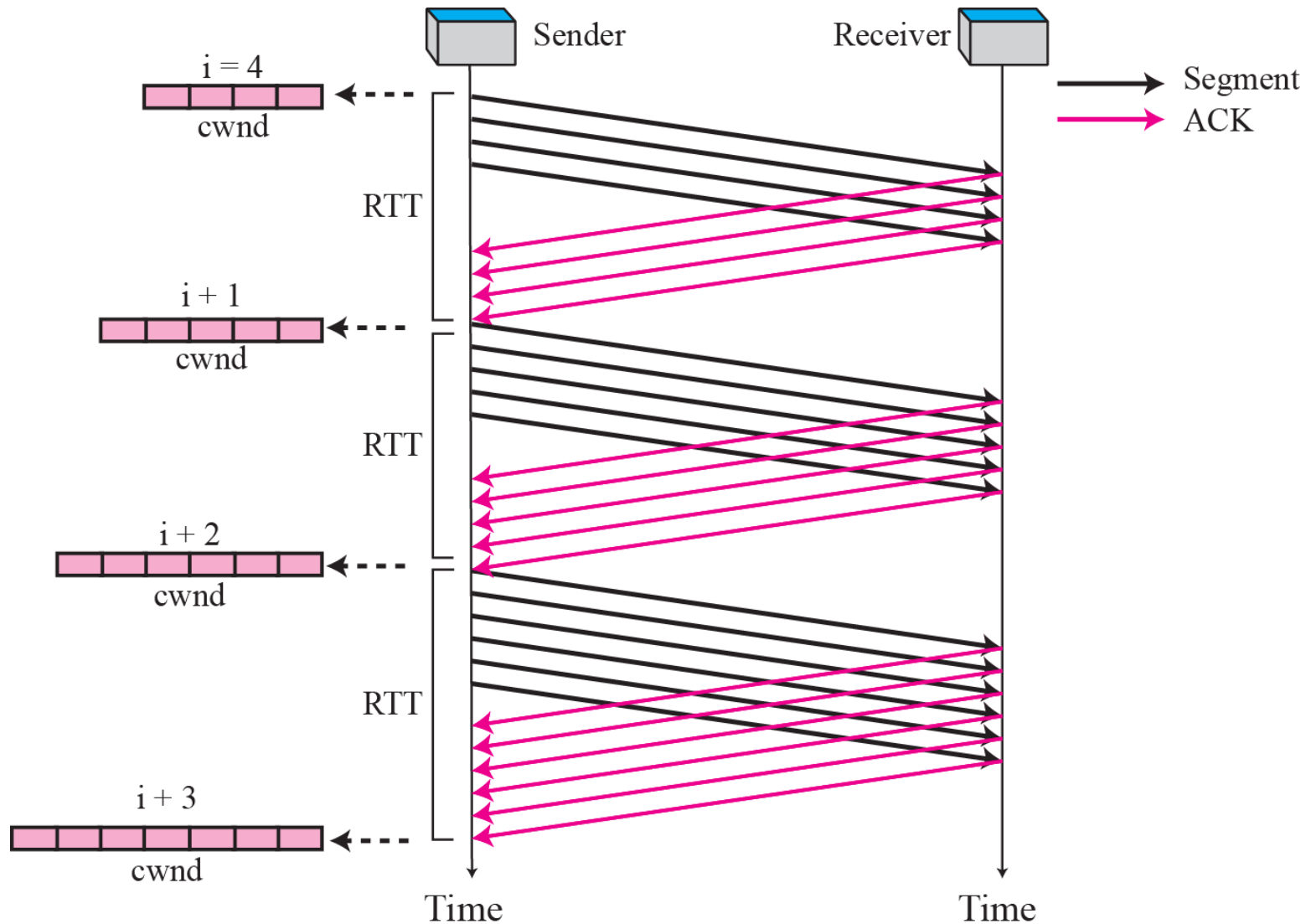
After 3 RTT

→ $\text{cwnd} = 4 \times 2 = 8 \rightarrow 2^3$

Slow start strategy is slower in case of delayed acknowledgement

Slow start cannot continue indefinitely and there is a threshold to stop this phase (*ssthresh*)

Congestion avoidance, additive increase



In the congestion avoidance algorithm the size of the congestion window increases additively until congestion is detected.

Start

→

$\text{cwnd} = i$

After 1 RTT

→

$\text{cwnd} = i + 1$

After 2 RTT

→

$\text{cwnd} = i + 2$

After 3 RTT

→

$\text{cwnd} = i + 3$

Here the increase is based on number of RTT, not on the number of arrived ACK's

Congestion Detection : Multiplicative Decrease

If congestion occurs, the congestion window size must be decreased.

sender sense about congestion

when there is the need to retransmit a segment

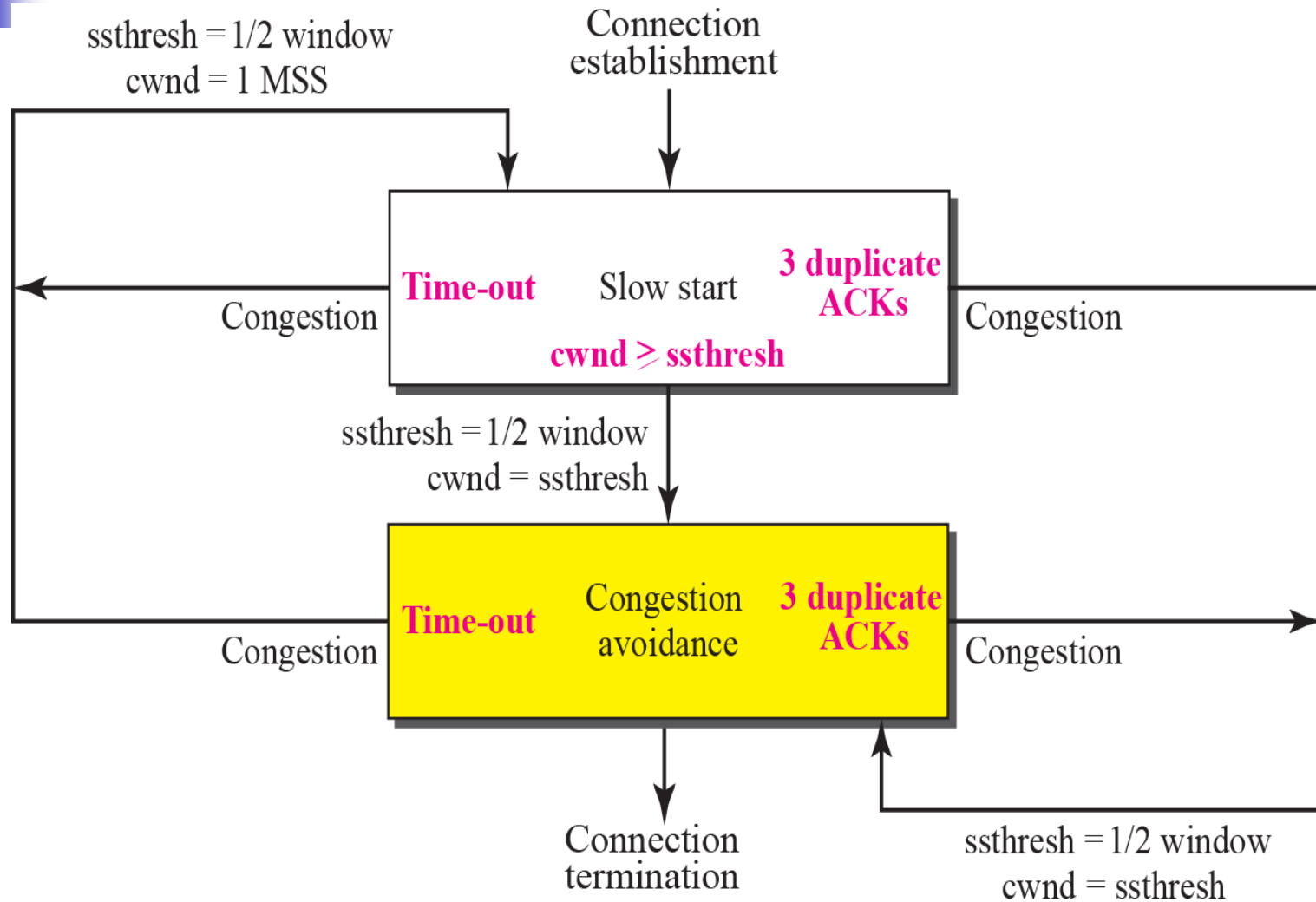
Retransmission occurs in two cases

- 1. RTO timer times out**
- 2. Three duplicate acknowledgement are received.**

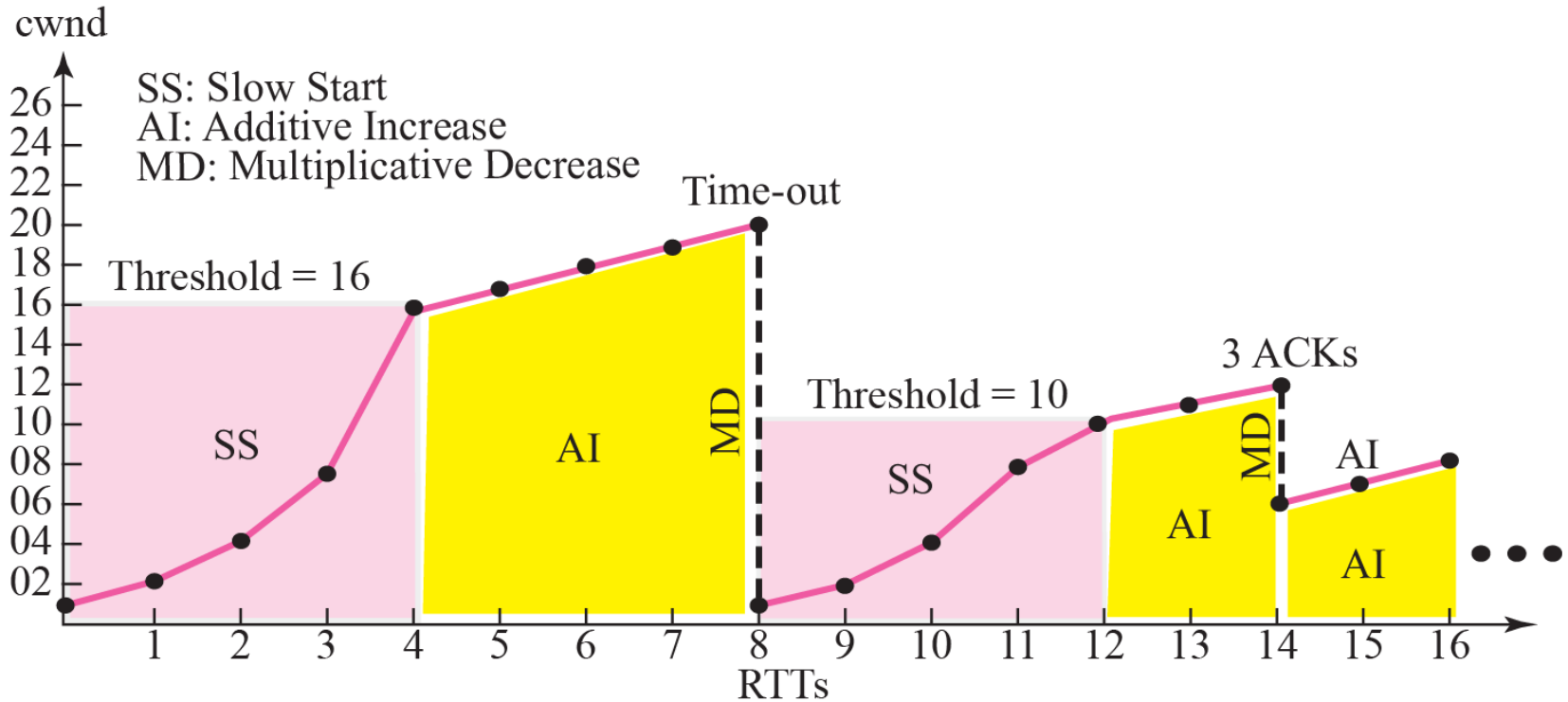
In both cases the size of threshold is dropped to half

1. If a time-out occurs, there is a stronger possibility of congestion; a segment has probably been dropped in the network and there is no news about the following sent segments. In this case TCP reacts strongly:
 - a. It sets the value of the threshold to half of the current window size.
 - b. It reduces cwnd back to one segment.
 - c. It starts the slow start phase again.
2. If three duplicate ACKs are received, there is a weaker possibility of congestion; a segment may have been dropped but some segments after that have arrived safely since three duplicate ACKs are received. This is called fast transmission and fast recovery. In this case, TCP has a weaker reaction as shown below:
 - a. It sets the value of the threshold to half of the current window size.
 - b. It sets cwnd to the value of the threshold (some implementations add three segment sizes to the threshold).
 - c. It starts the congestion avoidance phase.

TCP Congestion policy summary



Congestion example



Numericals

- Consider a TCP Congestion Policy, assuming the transmission from the slow start phase and the threshold at the start of the first transmission is 10 MSS. Assume that a time out occurs after the third transmission.
- Find the congestion window size at the end of the seventh transmission.

Numericals

- Consider a TCP Congestion Policy, assuming the transmission from the slow start phase and the threshold at the start of the first transmission is 10 MSS. Assume that a time out occurs after the third transmission.
- Find the congestion window size at the end of the seventh transmission.

Solution

1 Threshold at the beginning = 10 segments

Transmission Details

Threshold value after the timeout = 2

Cwnd at the 7th transmission = 4

Transmission Number	Cwnd (no. of segments)	Congestion status	Congestion phase
1	1	ok	1 st
2	2	ok	1 st
3	4	RTO expires	3 rd
4	1	ok	1 st
5	2	ok	1 st
6	3	ok	2 nd
7	4	ok	2 nd

- In a TCP connection, the initial sequence number (ISN) at the client site is 3,070. The client opens the connection, sends only one segment carrying 1,200 bytes of data, and closes the connection. What is the sequence number of segment sent by the client? (for practise)

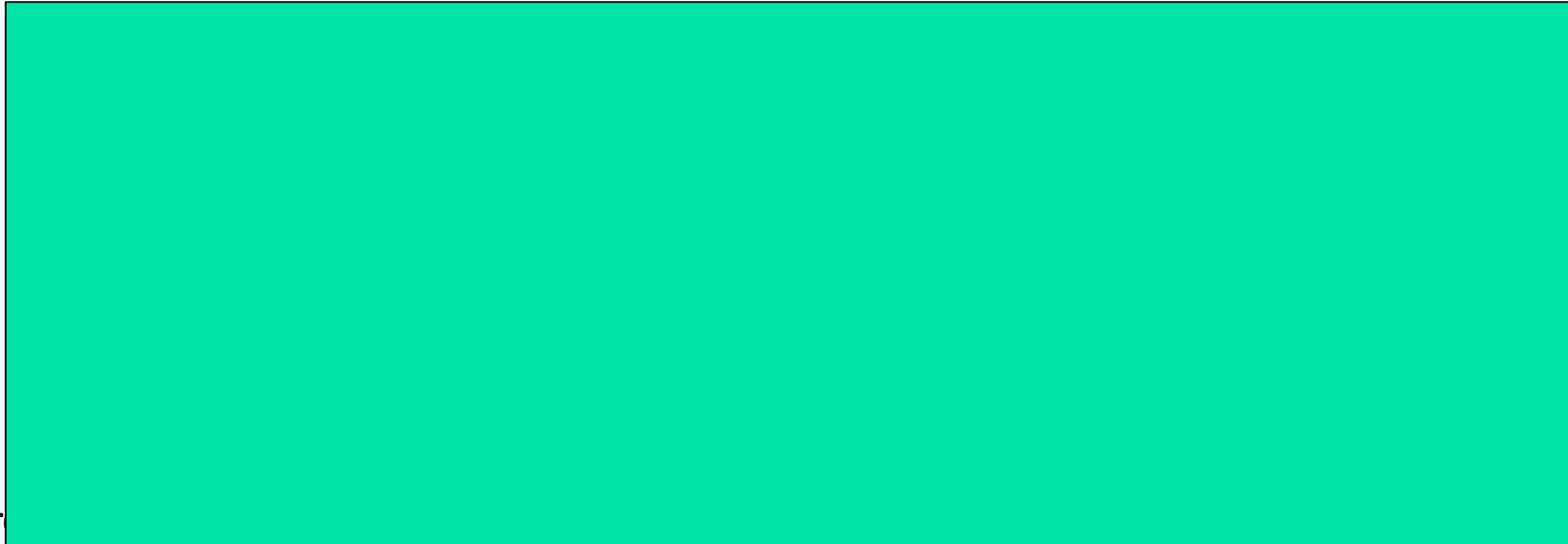
- In a TCP connection, the initial sequence number (ISN) at the client site is 3,070. The client opens the connection, sends only one segment carrying 1,200 bytes of data, and closes the connection. What is the sequence number of segment sent by the client? (for practise)

Solution

SYN packet consumes one sequence number, so the sequence number will be 3071

A host has no data to send. It receives the following segments at the times shown (hour:minute:second:milliseconds after midnight). Show the acknowledgments sent by the host.

- a. Segment 1 received at 0:0:0:000.
- b. Segment 2 received at 0:0:0:027.
- c. Segment 3 received at 0:0:0:400.
- d. Segment 4 received at 0:0:1:200.
- e. Segment 5 received at 0:0:2:508.



A host has no data to send. It receives the following segments at the times shown (hour:minute:second:milliseconds after midnight). Show the acknowledgments sent by the host.

- a. Segment 1 received at 0:0:0:000.
- b. Segment 2 received at 0:0:0:027.
- c. Segment 3 received at 0:0:0:400.
- d. Segment 4 received at 0:0:1:200.
- e. Segment 5 received at 0:0:2:508.

Segment number	Received at host	Acknowledgement Action (Rule number)
1	0:0:0:000.	No Ack (rule 2)
2	0:0:0:027.	Ack =3 (rule 3)
3	0:0:0:400	No Ack (rule 2) but Ack=4 sent after 500 ms (rule 2)
4	0:0:1:200	No Ack (rule 2) but Ack=5 sent after 500 ms (rule 2)
5	0:0:2:508	No Ack (rule 2)